

南開大學
Nankai University

计算机学院

《人工智能导论》课程实验报告

特征人脸识别

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

实验时间：2025 年 11 月 24 日 星期一

目录

1 问题重述 **2**

2 设计思想 **3**

2.1 向量化表示 3
2.2 中心化数据 3
2.3 计算协方差矩阵 3
2.4 特征值分解 3
2.5 选择前 K 个特征 3
2.6 降维与人脸识别 3
2.7 分类 4
2.8 人脸重建 4

3 程序代码 **4**

3.1 数据集的加载 4
3.2 训练特征脸算法的实现 5
3.3 人脸识别模型 6
3.4 计算识别准确率 6
3.5 人脸重建模型 7
3.6 重建训练集与测试集中的人脸 7

4 测试结果 **8**

4.1 模型测试结果 8
4.2 在线测试结果 9

5 总结 **9**

1 问题重述

实验背景

本实验采用特征脸（Eigenface）算法进行人脸识别。特征脸（eigenface）是第一种有效的人脸识别方法，通过在一大组描述不同人脸的图像上进行主成分分析（PCA）获得。本次实验要求大家构建一个自己的人脸库（建议）：大家可以选择基于 ORL 人脸库添加自己搜集到的人脸图像形成一个更大的人脸库，要求人脸库中的每一张图像都只包含一张人脸且眼睛的中心位置对齐（通过裁剪或缩放，使得每张人脸图像大小尺寸一致且人脸眼睛的中心位置对齐）。为了方便同学们操作，大家也可以选择直接基于 ORL 人脸库进行本次实验。

实验内容

在模型训练过程中，首先要根据测试数据求出平均脸，然后将前 K 个特征脸保存下来，利用这 K 个特征脸对测试人脸进行识别。此外，对于任意给定的一张人脸图像，可以使用这 K 个特征脸对原图进行重建。

实验要求

- 求解人脸图像的特征值与特征向量构建特征脸模型。
- 利用特征脸模型进行人脸识别和重建，比较使用不同数量特征脸的识别与重建效果。
- 使用 Python 语言。

参考资料

- 特征脸 - 维基百科
- Eigenface for Recognition
- ORL 人脸数据库
- The Yale Face Database

注意事项

- Python 与 Python Package 的使用方式，可在右侧 API 文档中查阅。
- 当右上角的「Python 3」长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角「Kernel」-「Restart Kernel」）。

2 设计思想

2.1 向量化表示

每一张人脸图像其实是一个二维的像素矩阵。为了使用机器学习算法（比如 PCA）处理这些图像，需要将每张二维图像转化为一个一维向量。假设每张图像的尺寸是 112×92 ，则每张图像可以表示为一个包含 10304 个元素（即 112×92 ）的一维向量。

2.2 中心化数据

在进行主成分分析（PCA）之前，首先要对图像数据进行中心化。中心化的目的是去除数据中的偏移，使得每个样本的均值为零。也就是对于每张图像 x_i ，我们需要从每个图像中减去训练集的平均脸 μ ：

$$x_i' = x_i - \mu$$

其中，μ 是所有训练图像的像素平均值（每个像素位置上所有图像的均值）。

2.3 计算协方差矩阵

中心化之后，图像的数据表示为矩阵 X ，其中每一行是一个人脸图像的向量。然后我们通过计算协方差矩阵来捕捉不同人脸之间的差异：

$$C = \frac{1}{N-1} X^T X$$

其中，X 是一个 $N \times D$ 的矩阵，N 是训练样本的数量，D 是每张图像的像素数（10304）。

2.4 特征值分解

对协方差矩阵 C 进行特征值分解，得到特征值和特征向量：

$$Cv_k = \lambda_k v_k$$

其中，λ_k 是特征值，表示每个特征的方差大小；v_k 是对应的特征向量，表示每个主成分（即特征脸）的方向。

2.5 选择前 K 个特征

通过对特征值的排序，我们选择前 K 个最大特征值对应的特征向量。这些特征向量组成了新的基空间，我们将人脸图像投影到这个新的基空间中。这些前 K 个特征向量即为特征脸，它们捕捉到了数据集中的主要特征。

2.6 降维与人脸识别

将每张人脸图像投影到前 K 个特征脸上，得到一个低维表示（即降维后的向量）：

$$z_i = W^T(x_i - \mu)$$

其中，W 是由前 K 个特征脸组成的矩阵，z_i 是图像 x_i 在特征脸空间中的表示。

2.7 分类

在识别阶段，我们将待识别人脸的投影向量与训练集中的投影向量进行比对，选择与训练集样本最相似的一个作为识别结果。

2.8 人脸重建

特征脸不仅可以用于人脸识别，还可以用于人脸重建。通过从特征脸空间中重构出图像，恢复出原始的人脸图像。重建公式如下：

$$\hat{x}_i = \mu + Wz_i$$

其中，$\hat{x}_i$ 是重建的人脸图像，W 是前 K 个特征脸组成的矩阵，z_i 是图像的特征表示。

本次实验比较简单，我们只需要按照上述思路补全三个函数即可完成实验。

3 程序代码

3.1 数据集的加载

```python
def spilt_data(nPerson, nPicture, data, label):
    """
    分割数据集

    :param nPerson : 志愿者数量
    :param nPicture: 各志愿者选入训练集的照片数量
    :param data : 等待分割的数据集
    :param label: 对应数据集的标签
    :return: 训练集，训练集标签，测试集，测试集标签
    """
    # 数据集大小和意义
    allPerson, allPicture, rows, cols = data.shape

    # 划分训练集和测试集
    train = data[:nPerson,:nPicture,:,:].reshape(nPerson*nPicture, rows*cols)
    train_label = label[:nPerson, :nPicture].reshape(nPerson * nPicture)
    test = data[:nPerson, nPicture:, :, :].reshape(nPerson*(allPicture - nPicture), rows*cols)
    test_label = label[:nPerson, nPicture:].reshape(nPerson * (allPicture - nPicture))

    # 返回：训练集，训练集标签，测试集，测试集标签
    return train, train_label, test, test_label

datapath = './ORL.npz'
ORL = np.load(datapath)
data = ORL['data']
label = ORL['label']

print("数据格式(志愿者数，各志愿者人脸数，height，width):", data.shape)
print("标签格式(志愿者数，各志愿者人脸数):", label.shape)

```

```
train_vectors, train_label, test_vectors, test_label = spilt_data(40, 5, data, label)
print("训练数据集:", train_vectors.shape)
print("测试数据集:", test_vectors.shape)
```

该部分完成了数据集的加载。我们通过路径 ORL.npz 读取数据，并将数据集划分为训练集和测试集。每个志愿者有 7 张图片，其中 5 张用于训练，剩下的 2 张用于测试。

我们获取了数据，得到不同角度的人脸图像并打印。



图 3.1.1

3.2 训练特征脸算法的实现

```
def eigen_train(trainset, k=20):
    avg_img = np.mean(trainset, axis=0)
    norm_img = trainset - avg_img
    feature = trainset[0:k]
    cov_matrix = np.dot(norm_img.T, norm_img)
    eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
    idx = np.argsort(-eigenvalues)
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]
    feature = eigenvectors[:, 0:min(k, eigenvectors.shape[1])].T
    return avg_img, feature, norm_img
```

训练步骤：

1. **中心化**：每个图像减去平均脸 (avg_img)，得到中心化图像 (norm_img)。
2. **协方差矩阵计算**：使用 np.dot(norm_img.T, norm_img) 计算协方差矩阵。
3. **特征值分解**：对协方差矩阵进行特征分解，得到特征值和特征向量，并按照特征值从大到小排序。
4. **选取前 K 个特征脸**：取出前 K 个特征向量作为特征脸。

输出两张提取到的特征人脸。

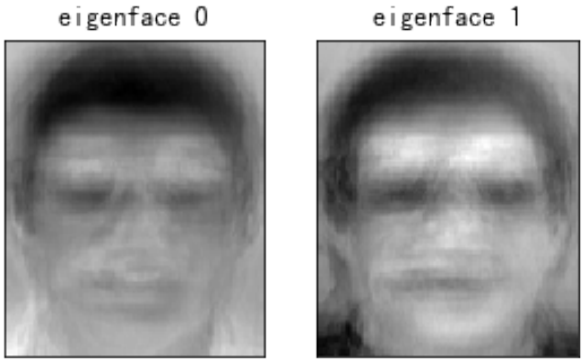


图 3.2.1

3.3 人脸识别模型

```python
def rep_face(image, avg_img, eigenface_vects, numComponents=0):
    """
    用特征脸（eigenface）算法对输入数据进行投影映射，得到使用特征脸向量表示的数据
    """
    numEigenFaces = min(numComponents, eigenface_vects.shape[0])
    w = eigenface_vects[0:numEigenFaces, ].T
    representation = np.dot(image - avg_img, w)
    return representation, numEigenFaces
```

识别步骤：

1. **中心化输入图像**：图像减去平均人脸 (avg_img) 得到差异。
2. **投影到特征脸空间**：通过与前 numEigenFaces 个特征脸的内积，将图像转换到特征脸空间。
3. **返回特征向量表示**：最终输出该图像的特征向量及使用的特征脸数量。

3.4 计算识别准确率

```python
train_reps = []
for img in train_vectors:
    train_rep, _ = rep_face(img, avg_img, eigenface_vects, num_eigenface)
    train_reps.append(train_rep)

num = 0
for idx, image in enumerate(test_vectors):
    label = test_labels[idx]
    test_rep, _ = rep_face(image, avg_img, eigenface_vects, num_eigenface)

    results = []
    for train_rep in train_reps:
        similarity = np.sum(np.square(train_rep - test_rep))
        results.append(similarity)
    results = np.array(results)

    if label == np.argmin(results) // 5 + 1:
```

```
        num += 1

print("人脸识别准确率: {}%".format(num / 80 * 100))
```

识别步骤:

1. **为每张训练图像生成特征向量**：使用 `rep_face` 将训练集每张图像投影到特征脸空间，得到其特征向量。
2. **测试集投影与训练集对比**：每张测试图像同样投影到特征脸空间，计算与训练集每个样本的欧氏距离，选择最近的训练样本作为识别结果。
3. **计算准确率**：通过与真实标签对比，计算正确率。

3.5 人脸重建模型

```
def recFace(representations, avg_img, eigenVectors, numComponents, sz=(112,92)):
    """
    利用特征人脸重建原始人脸
    """
    face = np.dot(representations, eigenVectors[0:numComponents, ]) + avg_img
    return face, 'numEigenFaces_{}'.format(numComponents)
```

重建步骤:

1. **重建图像**：通过特征向量 `representations` 和特征脸向量 `eigenVectors`，结合平均脸 `avg_img` 重建图像。
2. **输出重建结果**：将重建后的图像还原为 112×92 的二维形状，并返回。

3.6 重建训练集与测试集中的人脸

```
# 重建训练集人脸
image = train_vectors[100].copy()
faces = []
names = []
for i in range(20, 200, 20):
    representations, numEigenFaces = rep_face(image, avg_img, eigenface_vects, i)
    face, name = recFace(representations, avg_img, eigenface_vects, numEigenFaces)
    faces.append(face)
    names.append(name)

plot_gallery(faces, names, n_row=3, n_col=3)

# 重建测试集人脸
image = test_vectors[54].copy()
faces = []
names = []
for i in range(20, 200, 20):
    representations, numEigenFaces = rep_face(image, avg_img, eigenface_vects, i)
    face, name = recFace(representations, avg_img, eigenface_vects, numEigenFaces)
    faces.append(face)
    names.append(name)
plot_gallery(faces, names, n_row=3, n_col=3)
```

重建步骤：

1. **选择一张图像进行重建**：通过 `recFace` 将选择的图像使用不同数量的特征脸进行重建。
2. **展示不同特征脸数量的重建效果**：随着特征脸数量的增加，重建效果逐渐清晰。

用特征人脸还原初始人脸图像的效果：

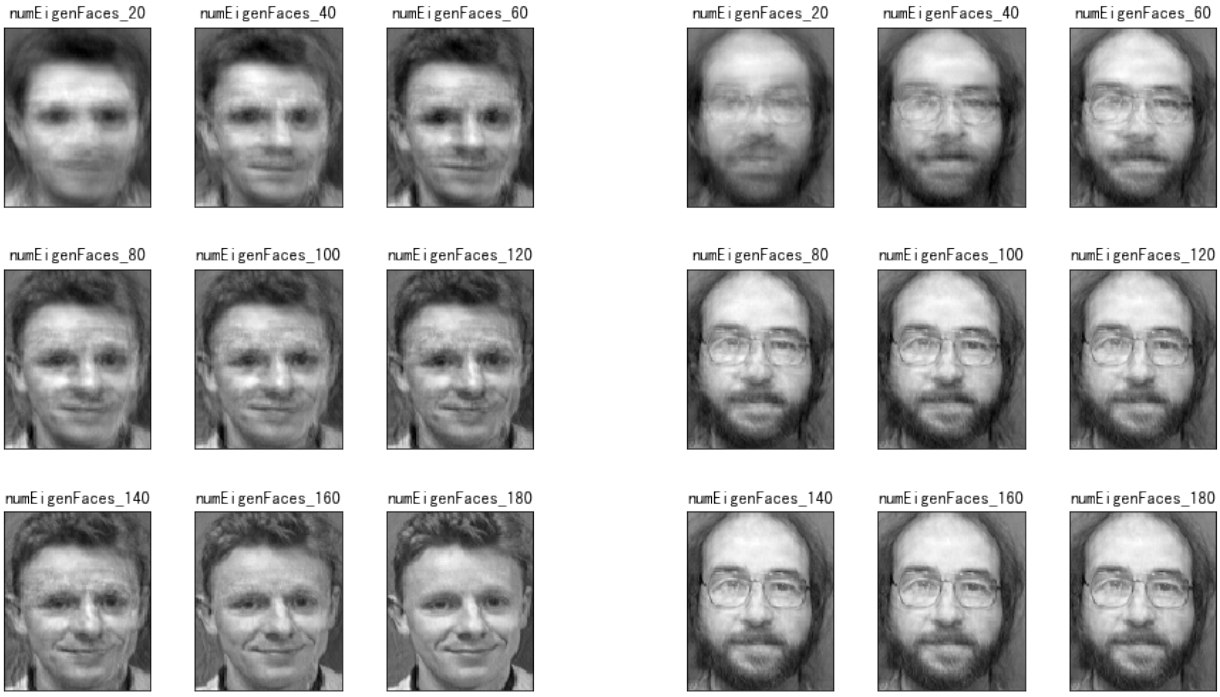


图 3.6.1

4 测试结果

4.1 模型测试结果

```
Python 3.7.5 (default, Dec  9 2021, 17:04:37)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.32.0 -- An enhanced Interactive Python. Type '?' for help.

[1]: !cd $WORK && python 'main.py'

训练数据集: (200, 10304)
测试数据集: (80, 10304)
Figure(360x240)
人脸识别准确率: 91.25%
重建训练集人脸
Figure(540x720)
------------------------------------------------------------
重建测试集人脸
Figure(540x720)
```

图 4.1.1: 模型测试结果

人脸识别准确率达到了 91.25

4.2 在线测试结果



图 4.2.1: 模型测试结果

5 总结

本实验虽然代码量不大，但函数实现难度感觉比较大，尤其是特征人脸的识别部分，我参考了 ai 生成结果和 CSDN 经验贴进行改进优化，最终准确率达到 91.25